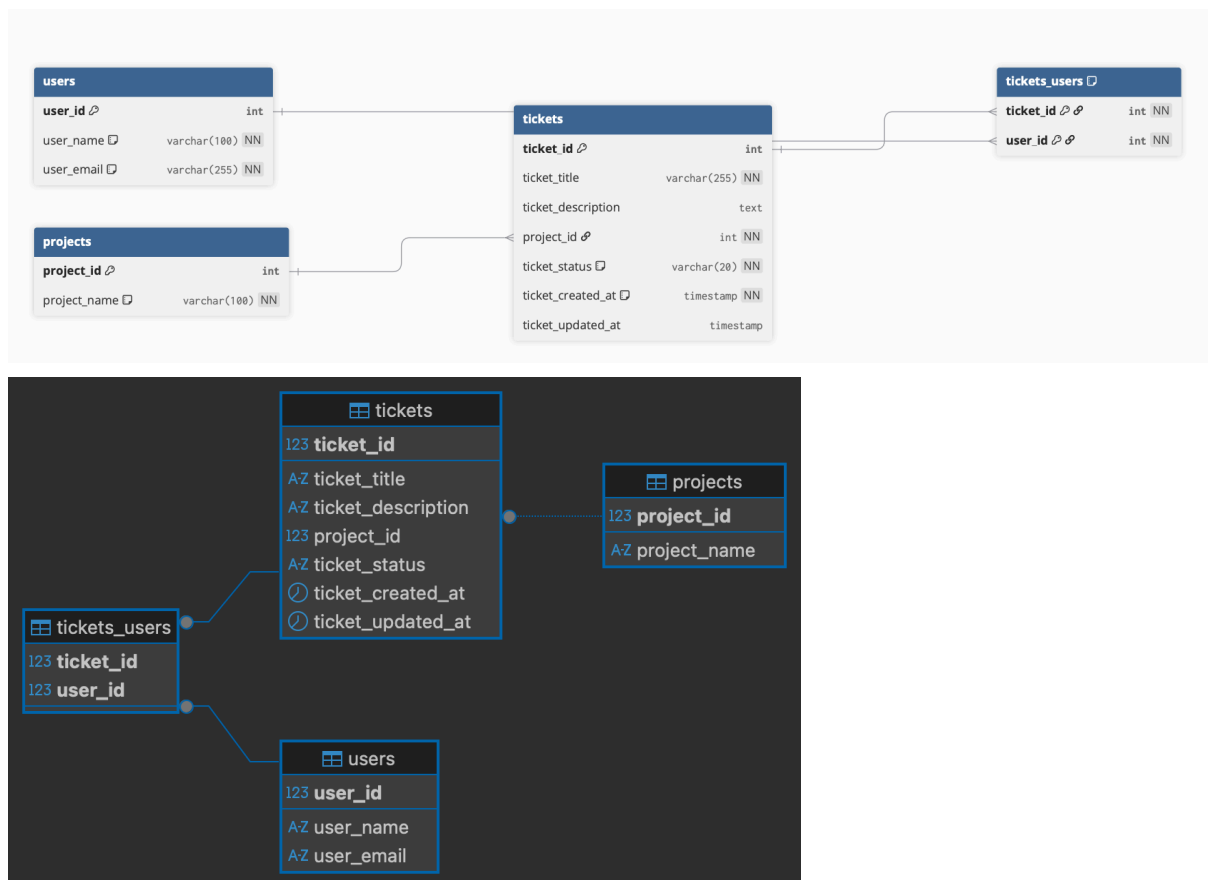


API Design document

1. Database diagram

Draw an ER diagram of your schema: tables, columns (with types), primary and foreign keys, and the relationships between them.

Suggested tool: dbdiagram.io — free, fast, and made for ER diagrams.



2. Endpoint summary

List every endpoint you plan to build in the following list:
(Replace the example endpoints with your own)

Method	Path	Description
GET	/users	Get all users
GET	/users/{id}	Get user by id

POST	/users	Create new user
PATCH	/users/{id}	Update user info by id
DELETE	/users/{id}	Remove user by id
GET	/projects	Get all projects with ticket counts per status
GET	/projects/{id}	Get project by id
POST	/projects	Create new project
PATCH	/projects/{id}	Update project name by id
DELETE	/projects/{id}	Remove project by id
GET	/tickets	Get all tickets. Filtered: ?text=, ?status=
GET	/tickets/{id}	Get ticket by id
POST	/tickets	Create new ticket
PATCH	/tickets/{id}	Update ticket info by id (title, description, status or project)
POST	/tickets/{id}/assignees	Add assignee to ticket
DELETE	/tickets/{id}/assignees/{userId}	Remove assignee from ticket

3. Endpoint description

In this section, list every endpoint in depth. The first endpoint is an example - remove it before submission.

/users

Get all users from DB.

Endpoint	/users
Method	GET
Response body	<pre>[{ "user_id": "integer", "user_name": "string", "user_email": "string" }]</pre>

/users/{id}

Get a single user by their ID from DB.

Endpoint	/users/{id}
Method	GET
Response body	<pre>{ "user_id": "integer", "user_name": "string", "user_email": "string" }</pre>
Validations	User with the given id must exist, otherwise return 404

/users

Create new user to the DB.

Endpoint	/users
Method	POST
Request body	<pre>{ "user_name": "string", "user_email": "string" }</pre>
Response body	<pre>{ "user_id": "integer", "user_name": "string", "user_email": "string" }</pre>
Validations	user_name and user_email are required, otherwise return 400 user_name must be at least 3 characters long, otherwise return 400 user_email must contain @ and be a valid email format, otherwise return 400

/users/{id}

Update an existing user's name or email.

Endpoint	/users/{id}
-----------------	-------------

Method	PATCH
Request body	<pre>{ "user_name": "string", "user_email": "string" }</pre>
Response body	<pre>{ "user_id": "integer", "user_name": "string", "user_email": "string" }</pre>
Validations	User with the given id must exist, otherwise return 404 At least one field (user_name or user_email) must be provided, otherwise return 400 If user_name is provided, it must be at least 3 characters long, otherwise return 400 If user_email is provided, it must be a valid email format with @, otherwise return 400

/users/{id}

Delete a user from the database.

Endpoint	/users/{id}
Method	DELETE
Response body	<pre>{ "message": "User with id {id} successfully deleted." }</pre>
Validations	User with the given id must exist, otherwise return 404

/projects

Get all projects, each with ticket counts per status.

Endpoint	/projects
Method	GET
Response body	<pre>[{ "project_id": "integer", "project_name": "string", "tickets": { "open": "integer", "in_progress": "integer", "closed": "integer" } }]</pre>

	]
--	---

/projects/{id}

Get a single project by its ID.

Endpoint	/projects/{id}
Method	GET
Response body	<pre>{ "project_id": "integer", "project_name": "string", "tickets": { "open": "integer", "in_progress": "integer", "closed": "integer" } }</pre>
Validations	Project with the given id must exist, otherwise return 404

/projects

Create a new project.

Endpoint	/projects
Method	POST
Request body	{ project_name: "string" }
Response body	<pre>{ "project_id": "integer", "project_name": "string" }</pre>
Validations	project_name is required and must be at least 3 characters long, otherwise return 400

/projects/{id}

Update an existing project's name.

Endpoint	/projects/{id}
Method	PATCH
Request body	{ project_name: "string" }
Response body	<pre>{ "project_id": "integer", "project_name": "string" }</pre>

Validations	Project with the given id must exist, otherwise return 404 project_name must be provided and not blank, also must be at least 3 characters long, otherwise return 400
--------------------	--

/projects/{id}

Delete a project and all its related tickets.

Endpoint	/projects/{id}
Method	DELETE
Response body	{ "message": "Project with id {id} successfully removed, and all related tickets to it too" }
Validations	Project with the given id must exist, otherwise return 404

/tickets/

Get all tickets. Supports optional search filters combined with AND logic.

Endpoint	/tickets
Method	GET
Query parameters	text - string - Matches against title or description; status - string - One of 'open', 'in_progress', 'closed'
Response body	[<pre> { "ticket_id": "integer", "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string", "ticket_created_at": "timestamp", "ticket_updated_at": "timestamp", "assignees": [{ "user_id": "integer", "user_name": "string" }] }] </pre>
Validations	If status is provided, it must be one of open , in progress , closed , otherwise return 400 No filters returns all tickets

/tickets/{id}

Get a single ticket with all its details.

Endpoint	/tickets/{id}
Method	GET
Response body	<pre>{ "ticket_id": "integer", "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string", "ticket_created_at": "timestamp", "ticket_updated_at": "timestamp", "assignees": [{ "user_id": "integer", "user_name": "string" }] }</pre>
Validations	Ticket with the given id must exist, otherwise return 404

/tickets

Create a new ticket.

Endpoint	/tickets
Method	POST
Request body	<pre>{ "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string" }</pre>
Response body	<pre>{ "ticket_id": "integer", "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string", "ticket_created_at": "timestamp", "ticket_updated_at": null, "assignees": [] }</pre>
Validations	ticket_title and project_id and ticket_status are required, otherwise return 400

	<code>ticket_description</code> is optional <code>ticket_status</code> must be one of <code>open</code>, <code>in progress</code>, <code>closed</code>, otherwise return <code>400</code> Project with the given <code>project_id</code> must exist, otherwise return <code>404</code> <code>ticket_created_at</code> is set automatically, <code>ticket_updated_at</code> is null on creation
--	--

/tickets/{id}

Update a ticket's title, description, status or project.

Endpoint	/tickets/{id}
Method	PATCH
Request body	<pre>{ "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string" }</pre>
Response body	<pre>{ "ticket_id": "integer", "ticket_title": "string", "ticket_description": "string", "project_id": "integer", "ticket_status": "string", "ticket_created_at": "timestamp", "ticket_updated_at": "timestamp", "assignees": [{ "user_id": "integer", "user_name": "string" }], "warning": "string null" }</pre>
Validations	Ticket with the given <code>id</code> must exist, otherwise return <code>404</code> At least one field must be provided, otherwise return <code>400</code> If <code>ticket_status</code> is provided, it must be one of <code>open</code>, <code>in progress</code>, <code>closed</code>, otherwise return <code>400</code> If <code>project_id</code> is provided, the project must exist, otherwise return <code>404</code> <code>ticket_updated_at</code> is set automatically to current timestamp <code>warning</code> is null when email was sent successfully. If email delivery fails, it will contain: "Ticket updated successfully, but email notification could not be sent."

/tickets/{id}/assignees

Add a user as an assignee to a ticket.

Endpoint	/tickets/{id}/assignees
Method	POST
Request body	<pre>{ "user_id": "integer" }</pre>
Response body	<pre>{ "ticket_id": "integer", "assignees": [{ "user_id": "integer", "user_name": "string" }], "warning": "string null" }</pre>
Validations	Ticket with the given id must exist, otherwise return 404 User with the given user_id must exist, otherwise return 404 User is not already assigned to this ticket, otherwise return 409 warning is null when email was sent successfully. If email delivery fails, it will contain: "Assignee added successfully, but email notification could not be sent."

/tickets/{id}/assignees/{user_id}

Remove an assignee from a ticket.

Endpoint	/tickets/{id}/assignees/{user_id}
Method	DELETE
Response body	<pre>{ "ticket_id": "integer", "assignees": [{ "user_id": "integer", "user_name": "string" }], "warning": "string null" }</pre>
Validations	Ticket with the given id must exist, otherwise return 404 User with the given user_id must exist, otherwise return 404 User is currently assigned to this ticket, otherwise return 404 warning is null when email was sent successfully. If email delivery fails, it will contain: "Assignee removed successfully,

but email notification could not be sent."

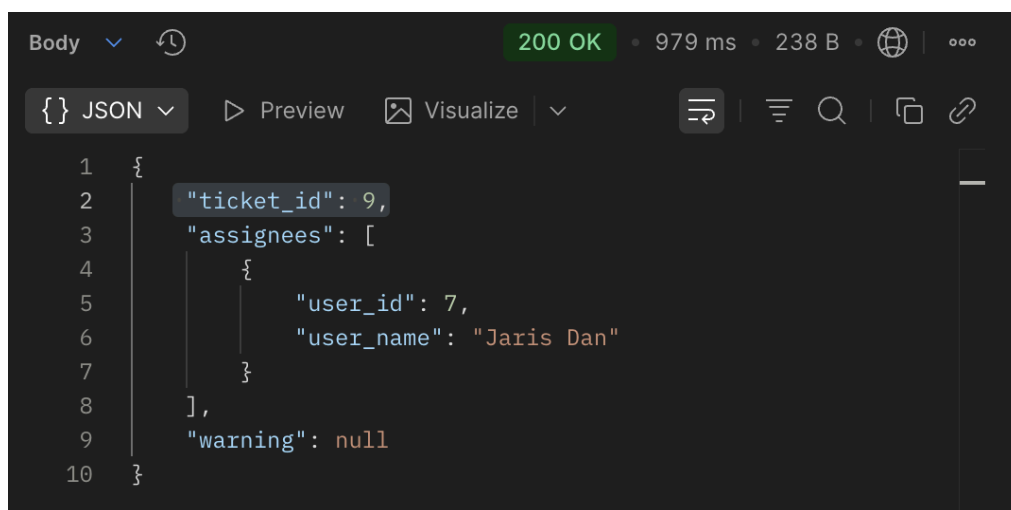
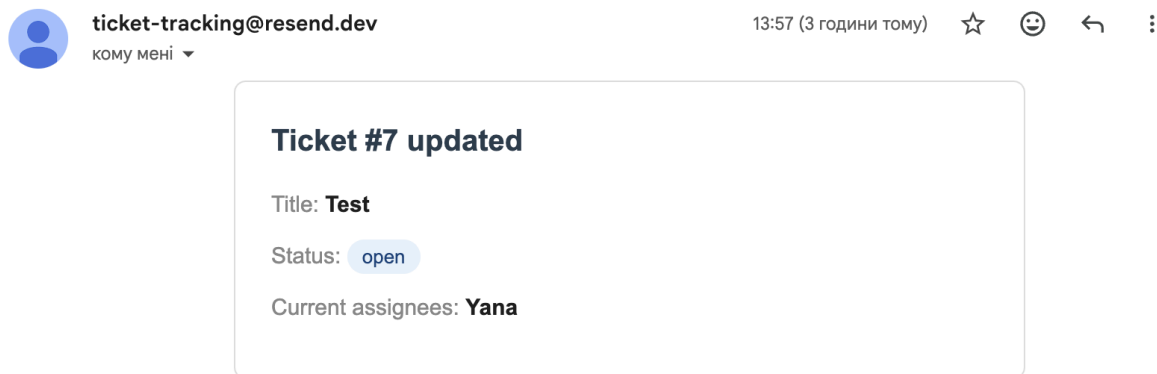
4. Email notifications

An email is sent each time a ticket is modified — on `PATCH /tickets/{id}`, `POST /tickets/{id}/assignees`, and `DELETE /tickets/{id}/assignees/{user_id}`.

All **current assignees** receive the email at the time of the operation. A newly added assignee is included in the notification. A removed assignee does **not** receive a notification — they are no longer part of the ticket.

The email contains the ticket ID, current title, current status, and the list of current assignees.

Email sending is handled synchronously via Resend API after a successful database operation. If sending fails, the request still returns 200 — the database change is not rolled back — but the response will include a `warning` field to inform the caller that the email notification could not be sent.



```
Body 200 OK • 861 ms • 545 B
{} JSON
1 {
2   "ticket_id": 4,
3   "ticket_title": "Java architecture",
4   "ticket_description": "Test",
5   "project_id": 2,
6   "ticket_status": "open",
7   "ticket_created_at": "2026-06-15T19:42:06.765078",
8   "ticket_updated_at": "2026-06-16T09:52:59.050058",
9   "assignees": [
10    {
11      "user_id": 3,
12      "user_name": "Alex Jim"
13    },
14    {
15      "user_id": 1,
16      "user_name": "Alla Melnic"
17    }
18  ],
19   "warning": "Ticket updated successfully, but email
20             notification could not be sent."
21 }
```